

19. Design: Top Level Approach to Development with Justification

19.1. In my first iteration, I will create the database required for my game solution.

WHY? I will need a database to store the information required for the game (for example user accounts, user scores for specific mazes, clue set files, maze data) and this is the fundamental part of creating my game. By creating this first, I can create a generic game database with the fields that I believe I will need and then I can build other areas of the game and link back to the database to enable the game to function. As I go through later iterations of the project, I can adapt and change the database structure to fit my solution as it develops. Creating the database will be my first iteration in my agile methodology.

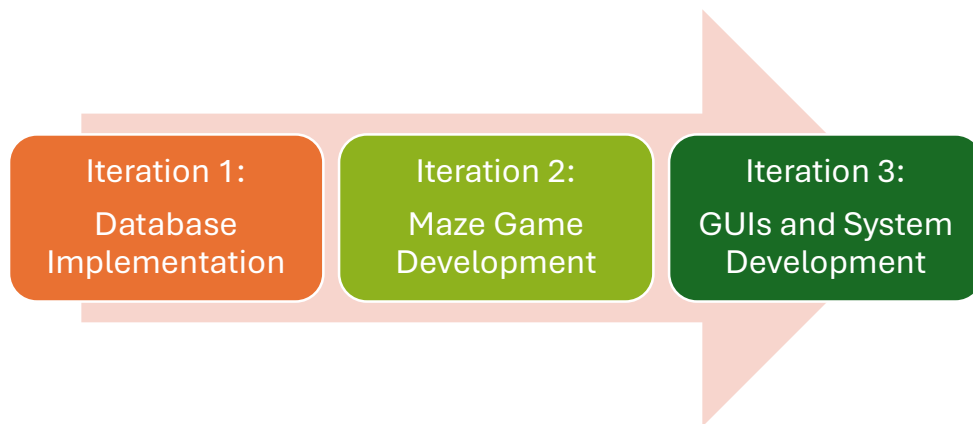
19.2. In my second iteration, I will create the main game, breaking it down into smaller areas, such as the characters and the maze.

WHY? I will create the main game as the second iteration to keep creation of the functionality needed for the gameplay separate to any real data that will need to be loaded in from the database. By doing this I can create a controlled environment with test variables without needing to access the database directly from the play game file during development.

19.3. In my third iteration, I will develop the GUIs and link them to each other to create the system surrounding the game, creating user accounts and saving their stores into the database. The first two iterations will provide the bulk of the processing, and this iteration will link these two areas together with user interfaces.

WHY? I will need to do the work in this order because I will need the main parts of the game (i.e. the database and the game play) to function before joining it together. By developing the GUIs last, I can build the final user interface system, linking it with the previous iterations to understand how it will work as a functional first prototype solution

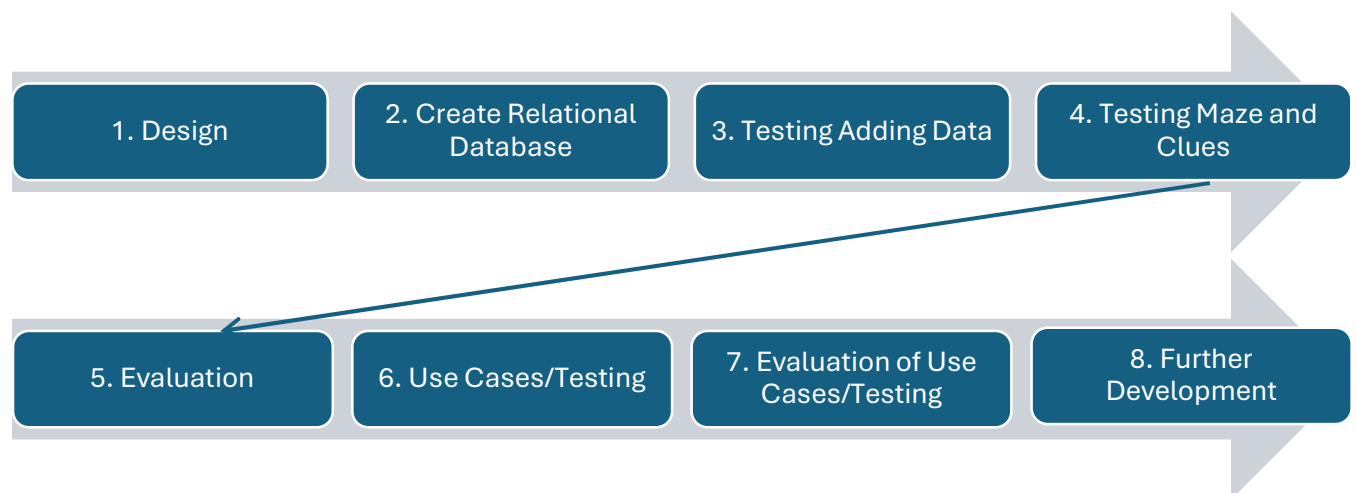
19.4. The following flow diagram shows the problem broken down into stages that are suitable for computational solutions:



20. Design: Overview of detail within each Iteration with Justification

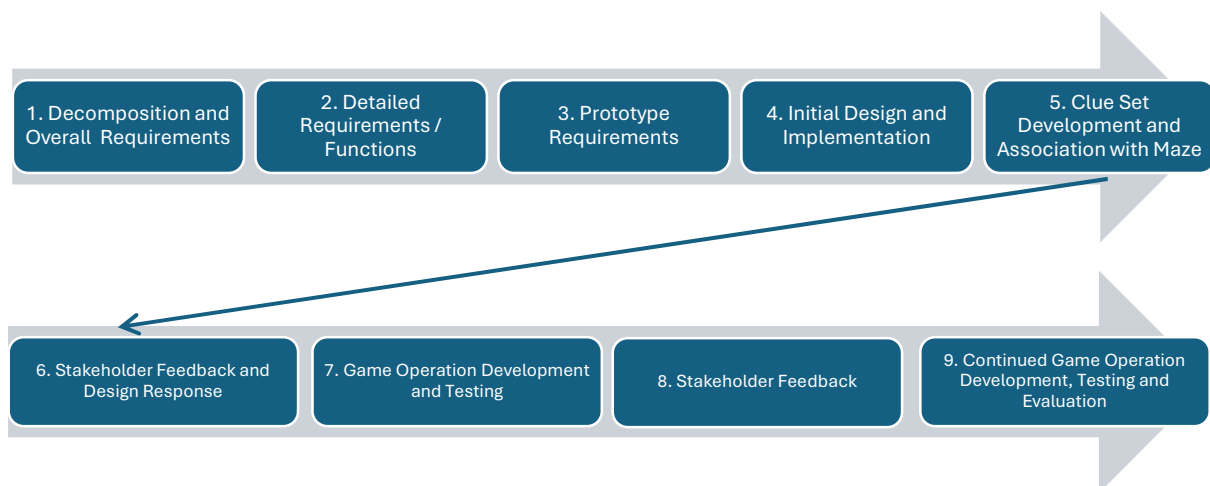
20.1. Having outlined the top level design above, I can define the content within each iteration that I will use to develop my game.

1st Iteration: Breakdown of Database Implementation



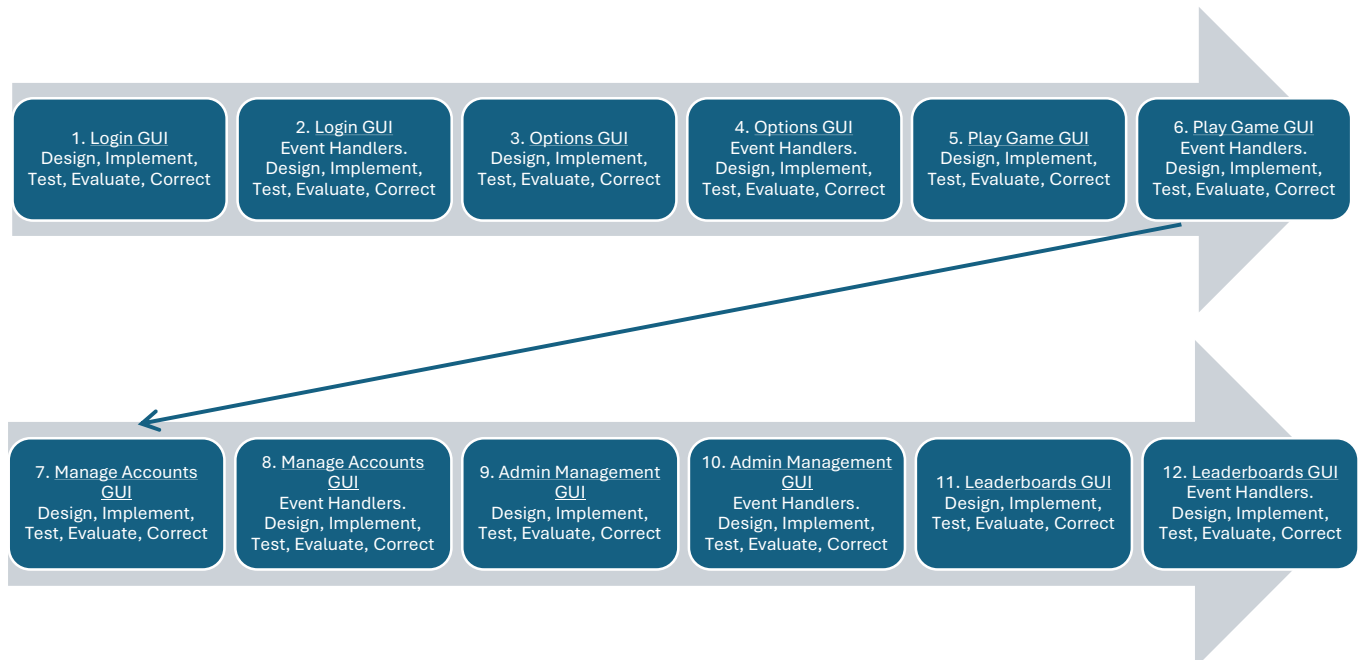
I have chosen this breakdown because it is easy to understand and includes all areas of the database which I believe will need to be developed. I have chosen the order of implementation phases above as I believe it is sequential and will provide me with a good framework to keep the project on track. I will use the bullets above as headings when I record this iteration.

2nd Iteration: Breakdown for Maze Game Development



I have chosen this breakdown to show the phase of work needed on the 2nd iteration of my project, which will focus on the development of the game system. As above, I have chosen the order of implementation phases that I believe will be sequential and provide me with a good framework to keep the project on track. I will use the bullets above as guidelines rather than as headings in section.

3rd Iteration: Breakdown for GUIs and System Development



I have chosen this breakdown to show the phase of work needed on the 3rd iteration of my project, which will focus on the multiple user interfaces needed for the game.. As above, I have chosen the order of implementation phases that I believe will be sequential and provide me with a good framework to keep the project on track. I will use the bullets above as headings in the section for the 3rd iteration.

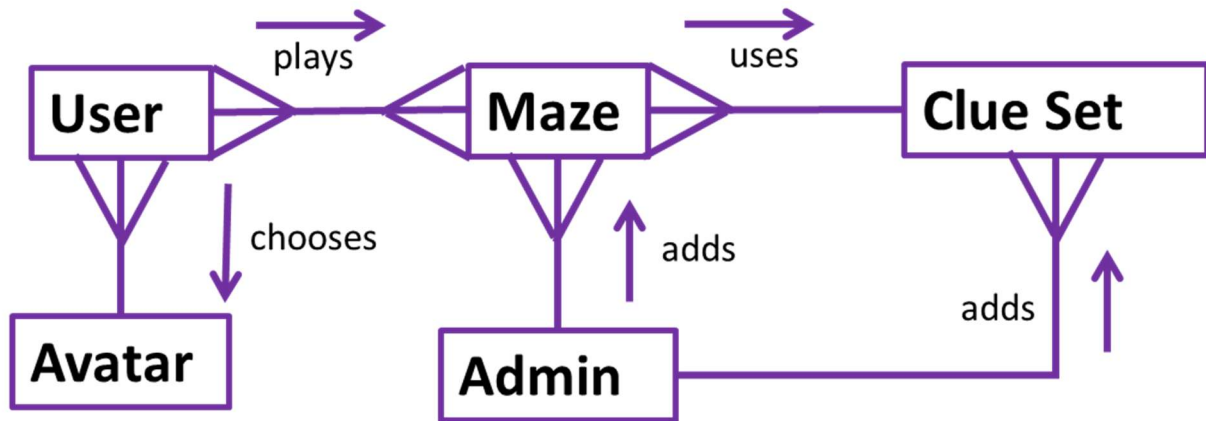
21. 1st Iteration: Database Development: Design

21.1. I have created an IPSO chart below to show Inputs, Processes, Storage and Outputs. This will help me to visualise what data is required, how it will be processed, what storage it might need and what outputs I should aim to achieve. In using this approach, I hope that it will prompt me to spot any gaps of things that need to be developed:

Input	Process	Storage	Output
User details: - User Name - User Avatar - User Account Type	Create Account Validate Input Editing Details	User Table (Database)	- Welcome Message - Create User Message - Different GUI for successful log in
Avatar Options - Avatar Filename	User Selects Avatar	Avatar Table (Database)	- Selected Avatar
Mazes - Maze File Name - Maze Difficulty - Maze Maximum Time	Load Selected Maze	Maze Table (Database)	- Selected Maze Game Plays
Clue Sets - Clue Set File Name - Clue Set Difficulty Level	Load Selected Clue Set	Clue Set Table (Database)	- Selected Clue Set Implemented in Selected Maze

21.2. Any inputs that are typed by the User will need validation – I will incorporate this during the 3rd iteration of this project where inputs will be validated before being added to the database.

21.3. The first diagram below is an entity relationship model to show the relationships that need to be accommodated within my database. I am using an entity relationship model to visualise the data to make the relationships that I need to create easier to understand and enable me to identify any problems with the data structure.



21.4. My Entity Relationship Model shows:

- A User can play one or more Mazes; the same Maze can be played by more than one user
- A Maze uses a selected Clue set; a Clue set can be assigned to many Mazes
- The User selects an Avatar when they create or manage their account
- An Admin can add Mazes and Clue sets. Note: it is likely the Admin will be included in the User entity

21.5. To help me to create an efficient database without data duplications, I am creating a normalised view of my database requirements (to third normalised form (3NF)). Below is the process that I went through to create a table to show what the database table would look like in first normal form (1NF):

First Normalised Form			Data duplication		No Atomicity	
Username	Avatar Filename	Account type	Maze filenames	Clueset filenames	Game stats	Game results
"Bob"	"Avatar1.txt"	"U"	"Maze1.txt"	"Clueset.txt"	"00:01:00", 1	23, "00:00:45"
"Bob"	"Avatar1.txt"	"U"	"Maze3.txt"	"Clueset.txt"	"00:05:00", 2	25, "00:02:45"
"Susan"	"Avatar3.txt"	"U"	"Maze1.txt"	"Clueset.txt"	"00:01:00", 1	17, "00:00:55"

But, having drawn the table above, I have realised that to make it into 1NF, I need to atomise the data and split out the data as shown below:

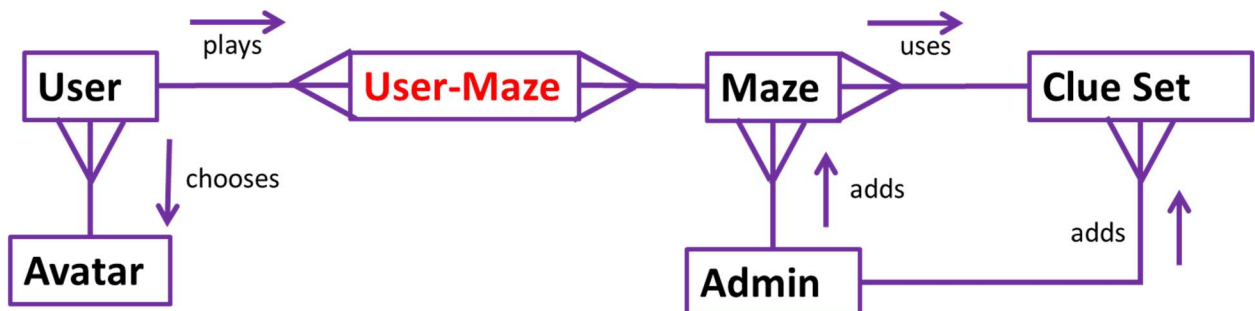
Username	Avatar Filename	Account type	Maze filenames	Clueset filenames	Game Max time	Game Difficulty	Game Score result	Game time result
"Bob"	"Avatar1.txt"	"U"	"Maze1.txt"	"Clueset.txt"	"00:01:00"	1	23	"00:00:45"
"Bob"	"Avatar1.txt"	"U"	"Maze3.txt"	"Clueset.txt"	"00:05:00"	2	25	"00:02:45"
"Susan"	"Avatar3.txt"	"U"	"Maze1.txt"	"Clueset.txt"	"00:01:00"	1	17	"00:00:55"

21.6. After Atomising the data, the table is now in 1NF but there are many data duplications due to the many to many relationships between the user information, for example:

Bob plays multiple games and the game with the maze file name

Maze1.txt is played by both Bob and Anna.

However, in databases, it is not efficient to have many to many relationships because it uses a lot of unnecessary storage, so my second diagram shows a more accurate entity relationship model for the tables needed in my database and their relation to each other. For an actual relational database, many-many relationships are not supported, so a link entity between User and Maze needs to be added:



- 21.7. To make the 1NF table above into 2NF form I need to split the table in half to remove partial dependencies. I have split the table into 2 tables called Maze and User with User_Maze as a link table between them. For now, I will ignore the result stats at the end of the table above. Below are the correctly assigned 2NF tables:

User

<u>UserID (PK)</u>	Username	Account Type	Avatar filename
1	"Bob"	"U"	"Avatar1.txt"
2	"Susan"	"U"	"Avatar3.txt"

Maze

<u>MazeID (PK)</u>	Maze Filename	Clueset Filename	Game Max time	Game Difficulty
1	"Maze1.txt"	"Clueset.txt"	"00:01:00"	1
2	"Maze3.txt"	"Clueset.txt"	"00:05:00"	2

User Maze

<i>UserID (FK)</i>	<i>MazeID (FK)</i>
1	1
1	2
2	1

At this point, the User_Maze table now holds 2 foreign keys changing the previous many to many relationship into a one to many and a many to one relationship. To add the game play statistics, I am going to include them in the User_Maze table and add an identifier field for each record as follows:

User Maze

<u>User_MazeID (PK)</u>	<i>UserID (FK)</i>	<i>MazeID (FK)</i>	Game Score	Game Time Taken
1	1	1	23	"00:00:45"
2	1	2	25	"00:02:45"
3	2	1	17	"00:00:55"

I also decided to add three additional fields NumEasy, NumMedium, NumHard to give Admins more of an insight into each the scores of each User. These are shown below:

NumEasy	NumMedium	NumHard
---------	-----------	---------

21.8. 3NF – to remove all dependencies

To turn the 2NF tables above into 3NF, I need to remove all dependencies to do this I have created 2 additional tables Avatar and Clueset which have connections in from the User and Maze table respectively using foreign keys. My complete set of 3NF tables are shown below:

User

<u>UserID (PK)</u>	Username	Account Type	AvatarID(FK)
1	"Bob"	"U"	1
2	"Susan"	"U"	3

Avatar

<u>AvatarID (PK)</u>	Avatar Filename
1	"Avatar1.txt"
2	"Avatar2.txt"
3	"Avatar3.txt"
4	"Avatar4.txt"

Maze

<u>MazeID (PK)</u>	Maze Filename	ClueID (FK)	Game Max time	Game Difficulty
1	"Maze1.txt"	1	"00:01:00"	1
2	"Maze3.txt"	1	"00:05:00"	2

Clueset

<u>ClueID (PK)</u>	Clueset filename
1	"Clueset.txt"

User_Maze

<u>User_MazeID (PK)</u>	UserID (FK)	MazeID (FK)	Game Score	Game Time Taken
1	1	1	23	"00:00:45"
2	1	2	25	"00:02:45"
3	2	1	17	"00:00:55"

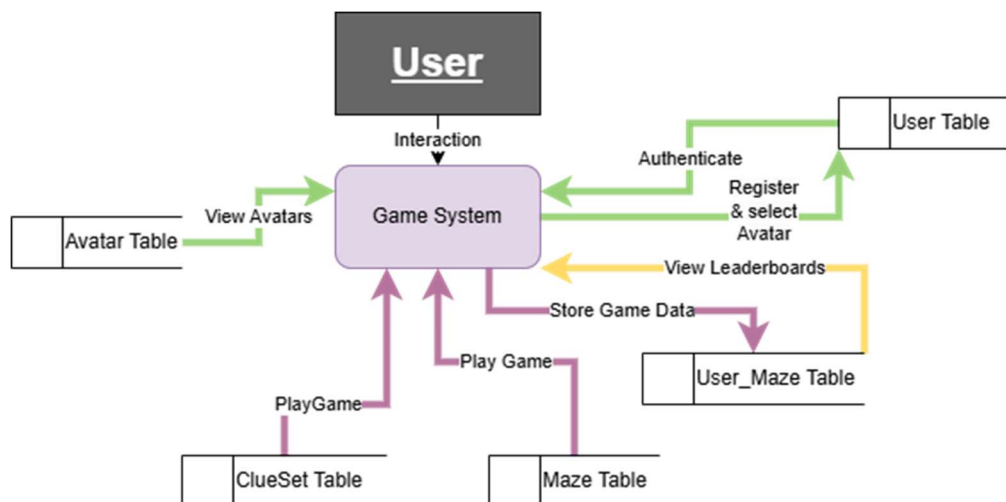
(including the additional 3 fields I mentioned earlier within the User_Maze table)

21.9. The following are the Entity descriptions with attributes for the 3NF tables shown above:

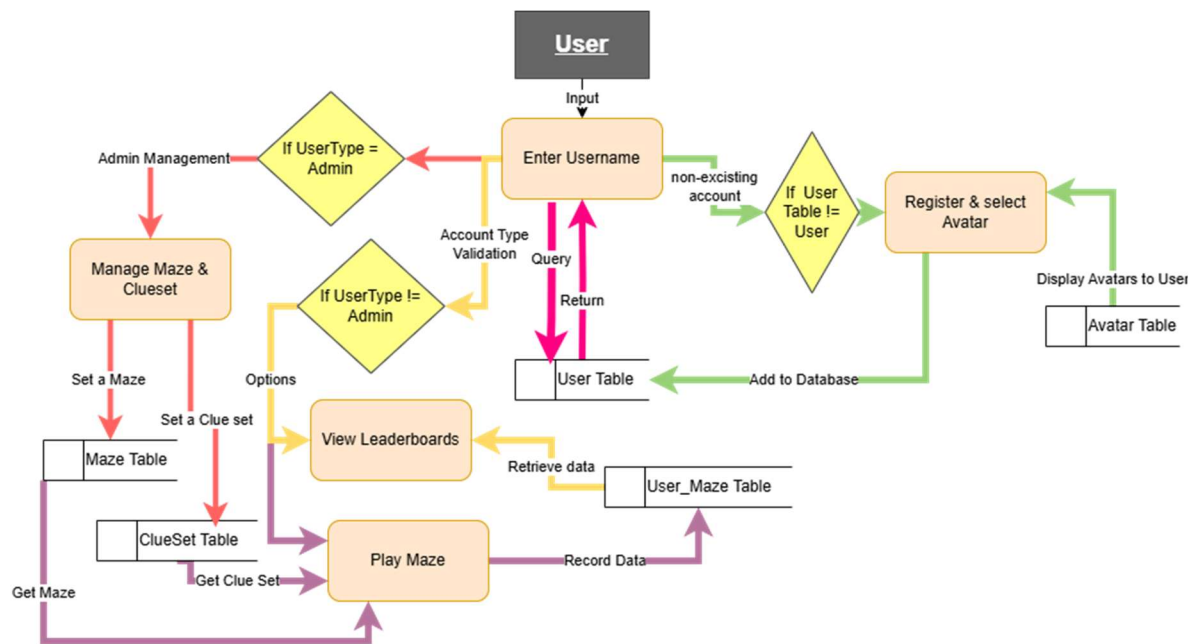
- User(UserID, Username, Account Type, *AvatarID*)
- Avatar(AvatarID, Avatar Filename)
- Maze(MazeID, Maze Filename, *ClueID*, Game Max Time, Game Difficulty)
- Clueset(ClueID, Clueset Filename)
- User_Maze(User_MazeID, *UserID*, *MazeID*, Game Score, Game Time Taken, NumEasy, NumMedium, NumHard)

21.10. To add detail to the entity relationship models shown above, I am creating two data flow diagrams to show how the data moves between process and how it is stored and retrieved.

- The first diagram that I have drawn is a Level 0 data flow diagram which is a simple data flow diagram showing the data flow from user's interaction into the game system and the subsequent data flow to and from each of the database tables. At Level 0, this first diagram does not go into detail about the system itself.



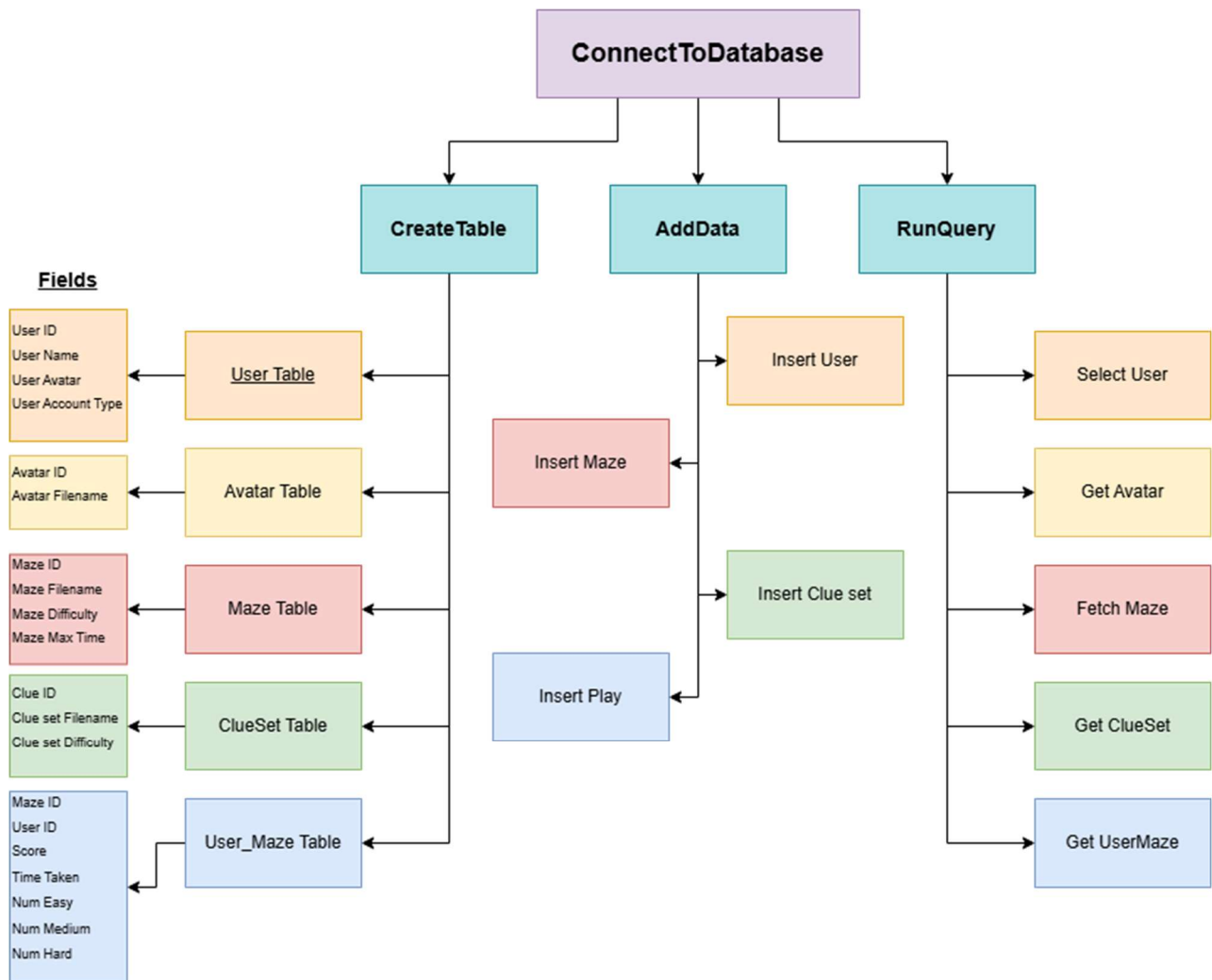
- The second diagram is a Level1 data flow diagram that provides more detail on how the data flows within the game system, and it shows decision points where processing occurs.



21.11. I am creating a Functions Table to identify the functions that I think I will need for this iteration, including appropriate names. I am doing this to give my solution structure and to minimise data input errors:

Type	Name	Purpose	Justification
Variable	DBFile	Stores the filename of the database and any information needed for the functionality of the game	By having the file as a variable the game will always open the same database file (all data stored won't be volatile)
Function	ConnectToDatabase	connects to the database using the SQLite module	Allows for successful data manipulation (adding and reading data) within the database.
Function	CreateTable	Creates the database tables if they do not already exist.	Enables the creation of the necessary tables
Function	AddData	Adds data into the database using DML scripts.	It is necessary to add data to the database to store the results of a game.
Function	RunQuery	Executes an SQL query and fetches the data from the appropriate location in the database	It is necessary to query the database in order to display previous results in a leaderboard and to test the game.

21.12. I am using a hierarchy chart (shown below) to link the functions above to the tables that I will use in my database, including the fields within the tables:



21.13. Having created the diagrams above to decompose the first iteration, I am moving on to creating table definitions for my database. I am doing this to outline the fields I will need within each table in my database. By creating these fields, I will have a structured idea of what content I need to have within each table so that the development of my database is smooth and compatible when I develop the rest of my game. I might add additional tables during development if I find these are insufficient.

21.14. The following shows my planned list of tables with justifications for their individual purposes.

Type	Name	Purpose	Justification
Table	Avatar	Records information of allowed avatars	It is necessary to store the available avatars that can be chosen
Table	User	Stores required user information	It is necessary to store all the user's details including their account type and a link to their avatar
Table	ClueSet	Stores required clue set filename	It is necessary to store all available Clueset files input
Table	Maze	Stores required Maze information including game difficulty and maximum time allowed	It is necessary to store the Maze information including a link to the set clue set
Table	User_Maze	Stores information of mazes played by each user.	It is necessary to store a user's scores for a specific game (linked to Maze and User Tables)

21.15. Below are the tables I will implement in my database, including the fields within them to store data.

USER

UserID (Primary Key)	Integer	Unique to each record in the table
Username	String	Unique to each record in the table since this is the log in the field
UserType	Char	To flag if a general user ('U') or Admin ('A')
AvatarID	Integer	Primary Key of Avatar table as a foreign key.

AVATAR

AvatarID (Primary Key)	Integer	To ensure all records in this table are unique
AvatarFilename	String	Reference to graphics file for Avatar.

MAZE

MazeID (Primary Key)	Integer	To ensure all records in this table are unique
MazeFilename	String	Reference to Maze definition text file
DifficultyLvl	Integer	1=Easy, 2=Medium, 3=Hard
MaxTime	Time	Max time allowed

CLUE SET

MazeID (Primary Key)	Integer	To ensure all records in this table are unique
MazeFilename	String	Reference to Maze definition text file
DifficultyLvl	Integer	1=Easy, 2=Medium, 3=Hard
MaxTime	Time	Max time allowed

USER-MAZE

UMID (Primary Key)	Integer	To ensure all records in this table are unique
UserID	Integer	Foreign Key to reference User
MazeID	Integer	Foreign Key to reference Maze
Score	Integer	Result of playing the maze game
TimeTaken	Time	How long the game took
NumEasy	Integer	Number of correctly answered Clue lvl 1
NumMedium	Integer	Number of correctly answered Clue lvl 2
NumHard	Integer	Number of correctly answered Clue lvl 3

22. 1st Iteration: Creating Relational Database: Development

22.1. In Year 12, I investigated implementing a flatfile log on a database¹² and developed subprograms to manage it (Appendix C). I will build on my existing knowledge and will modify and develop this to manage the **Relational Database** for this project. I will use CreateDatabase() as shown in the algorithm below to define the table and commit the changes to the database. This can be modified to include the new tables:

Procedure CreateDatabase(dbfile)

```
// procedure to define the required tables and commit to the dbfile passed as a parameter
// start with 'one end' tables – use their PK as FK
Avatar = CREATE TABLE IF NOT EXISTS Avatar(
    AvatarID integer primary key (autoincrement),
    Filename varchar(20) not null (mandatory))
User = CREATE TABLE IF NOT EXISTS User(
    UserID integer primary key (autoincrement),
    Username varchar(15) not null (mandatory),
    Type char not null (mandatory), // 'A' for Admin, 'U' for User
    AvatarID integer not null (mandatory) // could set a default to be changed
    AvatarID references Avatar(AvatarID)) // foreign key
ClueSet = CREATE TABLE IF NOT EXISTS ClueSet(
    ClueID integer primary key (autoincrement),
    Filename varchar(20) not null)
Maze = CREATE TABLE IF NOT EXISTS Maze(
    MazeID integer primary key (autoincrement),
    Filename varchar(20) not null,
    Difficulty integer not null,
    MaxTime time not null,
    ClueID integer not null,
    ClueID references ClueSet(ClueID)) // foreign key
User-Maze = CREATE TABLE IF NOT EXISTS User-Maze(
    UMID integer primary key (autoincrement),
    UserID integer
    MazeID integer
    UserID references User(UserID), // foreign key
    MazeID references Maze(MazeID), // foreign key
    Score integer, // not mandatory – added when game completed
    TimeTaken time,
    NumEasy integer,
    NumMedium integer,
    NumHard integer)
// defined tables so add to database
open(dbfile) // open nominated database file
add Avatar
add User
add ClueSet
add Maze
add User-Maze
commit to database
close(dbfile)
endprocedure
```

- 22.2. Due to the procedure shown above having no outputs, the only way to test it once coded is to run it and check that it has no errors. To run it, I also coded the function `ConnectToDatabase()`.

```
def ConnectToDatabase(dbfile):  
    # Function to connect to database  
    # Creates it if it does not exist  
    # Returns the connection object  
    # conn = a connection object  
    # curs = a cursor object  
  
    conn = sqlite3.connect(dbfile)    # create connector  
    curs = conn.cursor()              # create cursor  
  
    # return a list = one thing  
    # At this point, the database is OPEN  
    return [conn, curs]
```

- 22.3. This returns `conn`, `curs` objects to manipulate the database and will be used in many other subprograms. I developed this in my Y12 database work and I am going to use this function in this project as well.

- 22.4. After running this code, it returned an error which I resolved as follows:

The Error Message and Identifying the Error in the Code:

```
sqlite3.OperationalError: duplicate column name: ClueID
```

```
#link table:  
User_Maze = '''CREATE TABLE IF NOT EXISTS User_Maze(  
    UMID integer primary key autoincrement,  
    UserID REFERENCES User(UserID),  
    MazeID REFERENCES Maze(MazeID),  
    Score integer,  
    TimeTaken time,  
    NumEasy integer,  
    NumMedium integer,  
    NumHard integer  
)'''
```

I have defined the tables as designed but sqlite3 only needs the foreign key reference to add the field

Running the Code with the Error Resolved

```
#open the database and put the table in it
conn, curs = ConnectToDatabase(dbfile)

curs.execute(Avatar) #add tables to database
curs.execute(User)
curs.execute(ClueSet)
curs.execute(Maze)
curs.execute(User_Maze)

conn.commit() #update the database with the new table
conn.close() #close the database

print()
return #end procedure
```

Once defined the tables are added to the open database and committed(saved)

✓ Run

When CreateTable is run, there are no error messages

23. 1st Iteration: Database Development and Testing: Adding Data

23.1. Having established that the database runs correctly, I need to add test data by using a DML script. I can create the following algorithmic procedure to add data to the database:

```
procedure AddData(dbfile, aScript)
// procedure to execute aScript in the nominated dbfile
// open the database
// execute aScript
// close database
endprocedure
```

23.2. I can repurpose the AddData() procedure from my Y12 database work¹² as follows:

```
def AddUser(dbfile, name, pin):
    #procedure to create DML script to add record
    #(ID, name, pin) to User table

    aDML = '''INSERT INTO User(Username, UserPIN) VALUES("%s", "%s")''' % (name,
pin)

    #print(aDML)#print the DML statement

    #open the database
    conn, curs = ConnectToDatabase(dbfile)

    curs.execute(aDML) #add record
    conn.commit() #update the database

    conn.close() #close the database
    return #end procedure
```

Need to change the parameter list.

The DML script can be created outside the procedure

The rest of the code can be run as is

Need to change the name to AddData so it can be used for any table

```
def RunQuery(dbfile, aQry):
    #Function to execute aQry script on the database and return the Answer table

    conn, curs = ConnectToDatabase(dbfile) #call function
    curs.execute(aQry) #execute the query
    AnsTable= curs.fetchall() #fetch the results

    conn.close() #close the database
    return AnsTable #return the results
```

Can re-use the RunQuery() function as is to test that the data has been added correctly

23.3. I then tested the AddData and RunQuery subprogrammes

Data values to add (all valid):

– Avatar filename "Avatar01.gif"

```
#add some test data
aDML = '''INSERT INTO Avatar(AvatarFilename) VALUES("%s")''' %
("Avatar01.gif")

AddData(dbfile, aDML)
aSQL = "select * from Avatar"
print(RunQuery(dbfile, aSQL))
```

– Avatar filename "Avatar02.gif"

– Assuming primary key =2

– Username "GiraffeLeaf", using "Avatar01.gif" = 1

– Username "RedLamp67", using "Avatar02.gif" = 2

```
User = '''CREATE TABLE IF NOT EXISTS User(
UserID integer primary key autoincrement,
UserName text NOT NULL,
Type text NOT NULL,
AvatarID REFERENCES Avatar(AvatarID)
)'''
```

Calling the `AddData()` and `RunQuery()` subprogrammes to test data was added correctly

```
[(1, 'Avatar01.gif'), (2, 'Avatar01.gif'), (3, 'Avatar01.gif')]
```

I ran the program 3 times so the Avatar file appears 3 time but with its own unique primary key

```
File "/home/runner/Cyber-Quiz-Maze/main.py", line 123, in AddData
    curs.execute(aScript) #add record
sqlite3.OperationalError: table User has no column named Type
```

23.4. I then encountered an error when trying to add users, as follows:

Error with User table:

```
User = '''CREATE TABLE IF NOT EXISTS User(
UserID integer primary key autoincrement,
UserName text NOT NULL,
Type text NOT NULL,
AvatarID REFERENCES Avatar(AvatarID)
)'''
```

When this program is run it gives me the following error message below.
About the User

When trying to add the users, I get an error message saying that the Type field cannot be found. The problem is likely to be in the User table definition

```
File "/home/runner/Cyber-Quiz-Maze/main.py", line 123, in AddData
    curs.execute(aScript) #add record
sqlite3.OperationalError: table User has no column named Type
```

To overcome the error I used the reference below to assist me in learning about how to correctly format references to other tables within my table definitions.

<https://www.geeksforgeeks.org/sqlite-foreign-key/>¹³ - this confirmed that I need to define the user table correctly.

23.5. I was able to solve the error by using the learning in this reference: <https://www.geeksforgeeks.org/sqlite-foreign-key/>¹³. which helped me to get the user table working correctly. I was then able to run the code without getting an error message and I was able to move on to adding test users to the database.

Example from the reference

```
CREATE TABLE Department
(
  emp_deptID varchar(255) ,
  emp_dept_name varchar(255)
  FOREIGN KEY (emp_deptID) REFERENCES
  Employees(emp_deptID)
)
```

The User Table now works and the users have been added to the database.

Original: error was the line:

```
AvatarID REFERENCES Avatar(AvatarID)
```

User = '''CREATE TABLE IF NOT EXISTS User(
UserID integer primary key autoincrement,
UserName text NOT NULL,
Type text NOT NULL,
AvatarID integer NOT NULL,
FOREIGN KEY (AvatarID) REFERENCES Avatar(AvatarID)
)'''

23.6. I then applied the correction to the other tables as follows to fix the error and ensure the User table works correctly, to allow Users to be added to database:

Applying correction to other tables

```
ClueID integer NOT NULL,  
FOREIGN KEY (ClueID) REFERENCES ClueSet(ClueID)
```

```
UserID integer NOT NULL,  
MazeID integer NOT NULL,  
FOREIGN KEY (UserID) REFERENCES User(UserID),  
FOREIGN KEY (MazeID) REFERENCES Maze(MazeID)
```

Fixing other tables at the many end of the relationship for the correct foreign key definition.

Maze and User Maze tables respectively

```
INSERT INTO User(UserName, Type, AvatarID) VALUES("GiraffeLeaf", "U", 4)  
INSERT INTO User(UserName, Type, AvatarID) VALUES("RedLamp67", "U", 5)  
[(1, 'Avatar01.gif'), (2, 'Avatar02.gif')]
```

The User table now works correctly, and the two users are now able to be added to the database

24. 1st Iteration: Database Development and Testing: Maze and Clues

24.1. I tested inserting Clue Set filenames with auto PK and it added the Clue Set filenames correctly as follows:

The Clue Set filenames to be added to test database:

ClueFilename = "WW1.txt"

ClueFilename =
"Animals.txt"

ClueFilename =
"Cyber.txt"

```
INSERT INTO ClueSet(ClueFilename)
Values("WW1.txt")
INSERT INTO ClueSet(ClueFilename)
Values("Animals.txt")
INSERT INTO ClueSet(ClueFilename)
Values("Cyber.txt")
[(1, 'WW1.txt'), (2, 'WW1.txt'), (
3, 'Animals.txt'), (4, 'Cyber.txt'
)]
```

24.2. I then added the Maze Definitions with associated Foreign key and it worked correctly as follows:

Associated Mazes to be added to the database.

"Maze1.txt" has Difficulty 1(Easy).
MaxTime "00:10:00" [hh:mm:ss] and
has a clue set 4 (Cyber.txt)

"Maze2.txt" has Difficulty 3(Hard).
MaxTime "00:05:00" [hh:mm:ss] and
has a clue set 3 (Animals.txt)

"Maze3.txt" has Difficulty
2(Medium). MaxTime "00:80:00"
[hh:mm:ss] and has a clue set 2
(WW1.txt)

```
INSERT INTO Maze(MazeFilename, Difficulty
, MaxTime, ClueID) VALUES("Maze1.txt", 1,
"00:10:00", 4 )
[(1, 'Maze1.txt', 1, '00:10:00', 4), (2,
'Maze1.txt', 1, '00:10:00', 4)]
INSERT INTO Maze(MazeFilename, Difficulty
, MaxTime, ClueID) VALUES("Maze2.txt", 3
, "00:05:00", 3 )
[(1, 'Maze1.txt', 1, '00:10:00', 4), (2,
'Maze1.txt', 1, '00:10:00', 4), (3, 'Maze
2.txt', 3, '00:05:00', 3)]
INSERT INTO Maze(MazeFilename, Difficulty
, MaxTime, ClueID) VALUES("Maze3.txt", 2,
"00:80:00", 2 )
[(1, 'Maze1.txt', 1, '00:10:00', 4), (2,
'Maze1.txt', 1, '00:10:00', 4), (3, 'Maze
2.txt', 3, '00:05:00', 3), (4, 'Maze3.txt
', 2, '00:80:00', 2)]
```

25. 1st Iteration: Evaluation of Database Development and Testing So Far

- 25.1. The development, testing and debugging so far demonstrates the relational database is correctly created and populated for the following functions:
- A User can be associated with an Avatar.
 - An Admin person can add Clue Set and Mazes and associate them.
- 25.2. This means that the backbone of my programme is successfully in place and any errors discovered with use case examples have been resolved. At this point, I have worked on four of the tables referenced in the two bullets above. The next step is to work on the fifth “User_Maze” table to connect it all together and simulate users playing mazes and record their results. Once this is in place, I will build a leaderboard.

26. 1st Iteration: Develop and Test: Use Cases for Play and Leaderboard

- 26.1. I created the following Use Cases to develop and test the User_Maze link table:

Use Case 1

“RedLamp67” plays
“Maze3.txt” and
completes it in
00:09:27, with a score

The Use Case has been
implemented correctly and a
game record recorded.

```
aDML = '''INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID)
VALUES(42, "00:09:27", 2, 3)'''
AddData(DBFile, aDML)
```

```
aSQL = "select * from User_Maze"
print(RunQuery(dbfile, aSQL))
```

```
INSERT INTO User_Maze(Score, TimeTaken, U
serID, MazeID) VALUES(42, "00:09:27", 2,
3)
[(1, 42, '00:09:27', None, None, None, 2,
3)]
```

26.2. I then carried out further developing and testing of the User_Maze link table with five more use cases to gain outputs that show the use cases have been implemented correctly:

Use case 2: “RedLamp67” plays “Maze3.txt” and completes it in 01:22:22, with a score of 75

Use case 3: “RedLamp67” plays “Maze3.txt” and completes it in 00:16:23, with a score of 34

Use case 4: “GiraffeLeaf” plays “Maze3.txt” and completes it in 00:14:15, with a score of 57

```
INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(75, "01:22:22", 2, 3)
[(1, 42, '00:09:27', None, None, None, 2, 3), (2, 75, '01:22:22', None, None, None, 2, 3)]
INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(34, "00:16:23", 2, 3)
[(1, 42, '00:09:27', None, None, None, 2, 3), (2, 75, '01:22:22', None, None, None, 2, 3), (3, 34, '00:16:23', None, None, None, 2, 3)]
INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(57, "00:14:15", 1, 3)
[(1, 42, '00:09:27', None, None, None, 2, 3), (2, 75, '01:22:22', None, None, None, 2, 3), (3, 34, '00:16:23', None, None, None, 2, 3), (4, 57, '00:14:15', None, None, None, 1, 3)]
INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(21, "00:10:48", 1, 3)
[(1, 42, '00:09:27', None, None, None, 2, 3), (2, 75, '01:22:22', None, None, None, 2, 3), (3, 34, '00:16:23', None, None, None, 2, 3), (4, 57, '00:14:15', None, None, None, 1, 3), (5, 21, '00:10:48', None, None, None, 1, 3)]
INSERT INTO User_Maze(Score, TimeTaken, UserID, MazeID) VALUES(10, "00:01:19", 1, 3)
[(1, 42, '00:09:27', None, None, None, 2, 3), (2, 75, '01:22:22', None, None, None, 2, 3), (3, 34, '00:16:23', None, None, None, 2, 3), (4, 57, '00:14:15', None, None, None, 1, 3), (5, 21, '00:10:48', None, None, None, 1, 3), (6, 10, '00:01:19', None, None, None, 1, 3)]
```

The Use Cases have been implemented correctly.

26.3. In order to prove that I can develop a leaderboard in due course using this stored data, I would need to be able to query and find the ID of a specific User by using any of the other fields. To test this, I selected the username “GiraffeLeaf” and tested to find the UserID as follows.

```
#now query multiple tables
aQry = '''SELECT User.UserID
FROM User
WHERE User.UserName = "GiraffeLeaf"'''
```

```
WantedID = RunQuery(DBFile, aQry)
WantedID = WantedID[0][0]
```

[(1,)]

Finding the Id of ‘GiraffeLeaf’ returns a list with 1 tuple in it with one item in it. To access the item then index the list and the tuple.

- 26.4. Having successfully proved the ability to find the ID of “GiraffeLeaf”, in order to develop a leaderboard, I would want to find the statistics for all the mazes that “GiraffeLeaf” has played in the use cases that I created. The following is a two part solution to finding the statistics for all mazes played by one UserID. Here is my two part solution:

```
aQry = '''SELECT User_Maze.UserID,  
User_Maze.MazeID, User_Maze.Score,  
User_Maze.TimeTaken  
FROM User_Maze  
WHERE User_Maze.UserID = %i''' %WantedID
```

This has found all of the Stats of the games “GiraffeLeaf” has played in my Use Cases

```
[(1, 3, 57, '00:14:15'), (1, 3, 21, '00:10:48'), (1, 3, 10, '00:01:19')]
```

- 26.5. I wanted to see whether I could do the same procedure in one step rather than two steps, so I tried this:

```
aQry = '''SELECT User_Maze.UserID,  
User_Maze.MazeID, User_Maze.Score,  
User_Maze.TimeTaken  
FROM User, User_Maze  
WHERE User.UserName = "%s"'''  
%"GiraffeLeaf"
```

This is the solution I tried in order to find the stats in one step. However, it didn't work because it returned the entire user maze table!

```
[(2, 3, 42, '00:09:27'), (2, 3, 75, '01:22:22'), (2, 3, 34, '00:16:23'), (1, 3, 57, '00:14:15'), (1, 3, 21, '00:10:48'), (1, 3, 10, '00:01:19')]
```

- 26.6. The solution that I tried above to do the procedure in one step didn't work so I did this instead:

```
aQry = '''SELECT User_Maze.UserID,  
User_Maze.MazeID, User_Maze.Score,  
User_Maze.TimeTaken  
FROM User_Maze  
WHERE User_Maze.UserID = %i'''  
%WantedID[0][0]
```

This gives me the one step solution that I wanted.

- 26.7. A Leaderboard usually shows the results for all the players of a particular Maze. This is the process that I developed to prove that I could create a leaderboard for my game:

```
#find who played a particular maze "Maze3.txt"
aQry = '''SELECT Maze.MazeID
FROM Maze
WHERE Maze.MazeFilename = "Maze3.txt"'''

WantedID = RunQuery(dbfile, aQry) #[ (ID, ) ]

aQry = '''SELECT User_Maze.UserID, User_Maze.Score, User_Maze.TimeTaken
FROM User_Maze
WHERE User_Maze.MazeID = %i''' %WantedID[0][0]

print('\n Records for "Maze3.txt":')
GameRecords = (RunQuery(dbfile, aQry))
print(GameRecords)

print("\n players for those games:")

for player in GameRecords:      #[UserID, Score, TimeTaken]
    WantedID = player[0]        #get ID from tuple
    #query the user table for the username
    aQry = '''SELECT UserName
FROM User
WHERE UserID = %s''' %WantedID    #[ (name, ) ]

    print(RunQuery(dbfile, aQry))
```

Run a Query to find the maze id of a given maze.

Run a query to find the records of all those who have played it.

Iterating through the User table for the user.

The Printed Output is as displayed, and it does not identify the individual User IDs

There are 6 game records for 6 player records which each correspond with each other.

Records for "Maze3.txt":
[(2, 42, '00:09:27'), (2, 75, '01:22:22'), (2, 34, '00:16:23'), (1, 57, '00:14:15'), (1, 21, '00:10:48'), (1, 10, '00:01:19')]

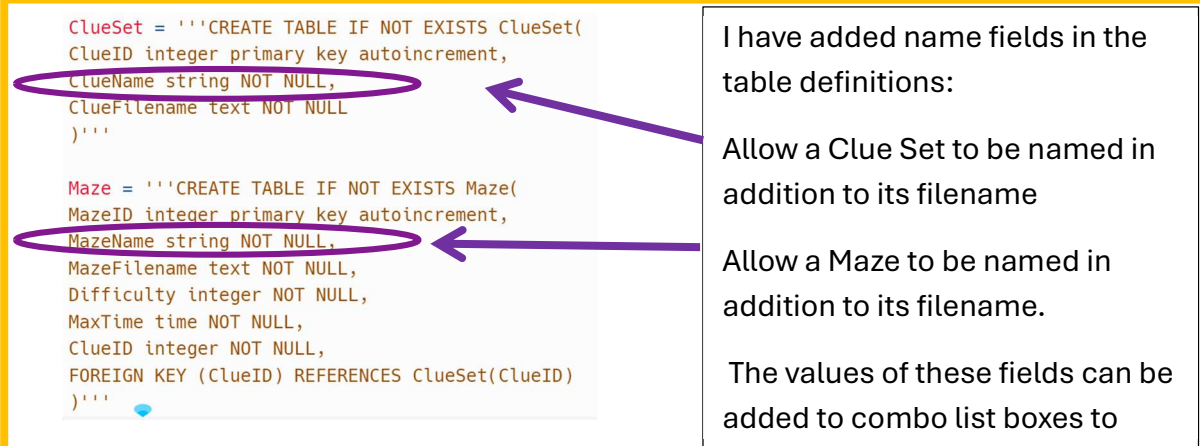
players for those games:
[('RedLamp67',)]
[('RedLamp67',)]
[('RedLamp67',)]
[('GiraffeLeaf',)]
[('GiraffeLeaf',)]
[('GiraffeLeaf',)]

27. 1st Iteration: Evaluation of Use Case Learning

- 27.1. The developments and tests above demonstrate that games can be recorded and interrogated for usernames so that leaderboards can be derived. It may be possible to add a new field name to give each maze a name (MazeName). It will be easier for the User to remember that they were playing a maze with a name rather than for example "Maze2.txt". The same is true for giving Avatars a name because an Avatar with a name will be easier for the User to remember than, for example "Avatar2.gif".

28. 1st Iteration: Continued Development of Leaderboard

28.1. I took the thinking from the evaluation above to add a name field for my maze and clueset tables as follows:



The image shows a screenshot of SQL code defining two tables: `ClueSet` and `Maze`. The code is as follows:

```
ClueSet = '''CREATE TABLE IF NOT EXISTS ClueSet(
ClueID integer primary key autoincrement,
ClueName string NOT NULL,
ClueFilename text NOT NULL
)'''

Maze = '''CREATE TABLE IF NOT EXISTS Maze(
MazeID integer primary key autoincrement,
MazeName string NOT NULL,
MazeFilename text NOT NULL,
Difficulty integer NOT NULL,
MaxTime time NOT NULL,
ClueID integer NOT NULL,
FOREIGN KEY (ClueID) REFERENCES ClueSet(ClueID)
)'''
```

Annotations in the image include:

- Two purple ovals highlighting `ClueName string NOT NULL,` in the `ClueSet` table definition and `MazeName string NOT NULL,` in the `Maze` table definition.
- Two purple arrows pointing from the text box on the right to these ovals.

The text box on the right contains the following text:

I have added name fields in the table definitions:

- Allow a Clue Set to be named in addition to its filename
- Allow a Maze to be named in addition to its filename.
- The values of these fields can be added to combo list boxes to